Отчёт по лабораторной работе №4

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **6**

2 Выполнение лабораторной работы **7**

2.1 Поэлементные операции над многомерными массивами 7
2.2 Транспонирование, след, ранг, определитель и инверсия матрицы 8
2.3 Вычисление нормы векторов и матриц, повороты, вращения . . . 10
2.4 Матричное умножение, единичная матрица, скалярное произведение 13
2.5 Факторизация. Специальные матричные структуры 14
2.6 Общая линейная алгебра 27
2.7 Задания для самостоятельного выполнения 28

3 Выводы **37**

Список иллюстраций

2.1 Поэлементные операции сложения и произведения 7
2.2 Средние значения 8
2.3 Создание массива и его транспонирование 8
2.4 След матрицы 9
2.5 Извлечение диагональных элементов как массив 9
2.6 Ранг матрицы 9
2.7 Инверсия матрицы 9
2.8 Определитель матрицы 10
2.9 Псевдобратная функция для прямоугольных матриц 10
2.10 Евклидова и p- нормы; Евклидово расстояние 11
2.11 Проверка по баз. опред. и угол между векторами 11
2.12 Евклидова и и p- нормы для 2мерной матрицы 11
2.13 Поворот на 180 градусов 12
2.14 Переворачивание строк 12
2.15 Переворачивание столбцов 12
2.16 Матрицы со случ. цел. значениями 13
2.17 Произведение матриц 13
2.18 Единичная матрица 13
2.19 Скалярное произведение векторов X и Y 14
2.20 Квадратная матрица 3x3 15
2.21 Единичный вектор 15
2.22 Вектор b 15
2.23 Решение исходного уравнения 16
2.24 LU-факторизация 16
2.25 Матрица перестановок 16
2.26 Вектор перестановок 17
2.27 Матрица L 17
2.28 Матрица U 17
2.29 Решение СЛАУ через матрицу A 17
2.30 Решение через объект факторизации 18
2.31 Детерминант матрицы A 18
2.32 QR-факторизация 18
2.33 Матрица Q 19
2.34 Матрица R 19
2.35 Проверка, что матрица Q - ортогональная 19
2.36 Симметризация матрицы A 19
2.37 Спектральное разложение симметризованной матрицы 20

2.38 Собственные значения 20
2.39 Собственные векторы 20
2.40 Проверка, что получится единичная матрица 21
2.41 Симметризация матрицы 22
2.42 Проверка симметричности 22
2.43 Добавление шума и проверка симметр-ти 23
2.44 Явное указание симметричности 24
2.45 Оценка эффективности для симметризованной матрицы 25
2.46 Оценка эффективности для зашумлённой матрицы 25
2.47 Оценка эффективности для сим. зашумлённой матрицы 25
2.48 Трёхдиагональная матрица 26
2.49 Оценка эффективности выполнения 26
2.50 Матрица с рациональными элементами 27
2.51 Единичный вектор 27
2.52 Вектор b 28
2.53 Решение с помощью функции \ 28
2.54 LU-разложение 28
2.55 Задание №1 29
2.56 Задание №2 29
2.57 Задание №3.a 29
2.58 Задание №3.b 30
2.59 Задание №4.a 30
2.60 Задание №4.b 30
2.61 Задание №4.c 31
2.62 Задание №4.d 31
2.63 Задание №5 32
2.64 Задание №5 32
2.65 Задание №6.a 32
2.66 Задание №6.b 33
2.67 Задание №6.c 33
2.68 Задание №7 34
2.69 Задание №7 34
2.70 Задание №8.a 35
2.71 Задание №8.b 36
2.72 Задание №8.c 36

Список таблиц

1 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2 Выполнение лабораторной работы

2.1 Поэлементные операции над многомерными массивами

Для матрицы 4 × 3 рассмотрим поэлементные операции сложения и произведения её элементов:

```
[10]: # Массив 4x3 со случайными целыми числами (от 1 до 20):
a = rand(1:20,(4,3))
# Поэлементная сумма:
println("Поэлементная сумма = ", sum(a))
# Поэлементная сумма по столбцам:
println("Поэлементная сумма по столбцам = ", sum(a,dims=1))
# Поэлементная сумма по строкам:
println("Поэлементная сумма по строкам = ", sum(a,dims=2))
# Поэлементное произведение:
println("Поэлементное произведение = ", prod(a))
# Поэлементное произведение по столбцам:
println("Поэлементное произведение по столбцам = ", prod(a,dims=1))
# Поэлементное произведение по строкам:
println("Поэлементное произведение по строкам = ", prod(a,dims=2))
```

```
Поэлементная сумма = 129
Поэлементная сумма по столбцам = [41 47 41]
Поэлементная сумма по строкам = [33; 31; 44; 21;;]
Поэлементное произведение = 227412541440
Поэлементное произведение по столбцам = [6048 6528 5760]
Поэлементное произведение по строкам = [648; 680; 3072; 168;;]
```

Рис. 2.1: Поэлементные операции сложения и произведения

Для работы со средними значениями можно воспользоваться возможностями

пакета Statistics:

```
[11]: # Подключение пакета Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics
# Вычисление среднего значения массива:
println("Среднее значение массива = ", mean(a))
# Среднее по столбцам:
println("Среднее по столбцам = ", mean(a,dims=1))
# Среднее по строкам:
println("Среднее по строкам = ", mean(a,dims=2))
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
Среднее значение массива = 10.75
Среднее по столбцам = [10.25 11.75 10.25]
Среднее по строкам = [11.0; 10.333333333333334; 14.666666666666666; 7.0;;]
```

Рис. 2.2: Средние значения

2.2 Транспонирование, след, ранг, определитель и инверсия матрицы

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) LinearAlgebra:

```
[12]: # Подключение пакета LinearAlgebra:
import Pkg
Pkg.add("LinearAlgebra")
using LinearAlgebra
# Массив 4x4 со случайными целыми числами (от 1 до 20):
b = rand(1:20,(4,4))
# Транспонирование:
transpose(b)
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
[12]: 4×4 transpose(::Matrix{Int64}) with eltype Int64:
  6  10  18   5
 11  18  11  11
 18   2  12   7
 17  10  11  16
```

Рис. 2.3: Создание массива и его транспонирование

```
[13]: # След матрицы (сумма диагональных элементов):
tr(b)
```

```
[13]: 52
```

Рис. 2.4: След матрицы

```
[14]: # Извлечение диагональных элементов как массив:
diag(b)
```

```
[14]: 4-element Vector{Int64}:
 6
 18
 12
 16
```

Рис. 2.5: Извлечение диагональных элементов как массив

```
[15]: # Ранг матрицы:
rank(b)
```

```
[15]: 4
```

Рис. 2.6: Ранг матрицы

```
[16]: # Инверсия матрицы (определение обратной матрицы):
inv(b)
```

```
[16]: 4×4 Matrix{Float64}:
 -0.0656058  -0.0307091    0.0862647    0.0295924
  0.0625349   0.110832    -0.0552764   -0.0977108
  0.103015    0.00921273  -0.00921273  -0.108878
 -0.06756    -0.0706309    0.0150754    0.168063
```

Рис. 2.7: Инверсия матрицы

```
[17]: # Определитель матрицы:
      det(b)

[17]: -21492.000000000004
```

Рис. 2.8: Определитель матрицы

```
[18]: # Псевдобратная функция для прямоугольных матриц:
      pinv(b)

[18]: 4×4 Matrix{Float64}:
       -0.0656058  -0.0307091    0.0862647    0.0295924
        0.0625349   0.110832    -0.0552764   -0.0977108
        0.103015    0.00921273  -0.00921273  -0.108878
       -0.06756    -0.0706309    0.0150754    0.168063
```

Рис. 2.9: Псевдобратная функция для прямоугольных матриц

2.3 Вычисление нормы векторов и матриц, повороты, вращения

Для вычисления нормы используется LinearAlgebra.norm(x).

Евклидова норма:

$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2}$

p-норма:

$\|\vec{A}\|_p = \left(\sum_{i=1}^{n} |a_i|^p\right)^{1/p}$

```
[19]: # Создание вектора X:
X = [2, 4, -5]
# Вычисление евклидовой нормы:
println("Евклидова норма = ", norm(X))
# Вычисление p-нормы:
p = 1
println("p-норма = ", norm(X,p))
# Расстояние между двумя векторами X и Y:
X = [2, 4, -5];
Y = [1,-1,3];
println("Расстояние между двумя векторами = ", norm(X-Y))
```

```
Евклидова норма = 6.708203932499369
p-норма = 11.0
Расстояние между двумя векторами = 9.486832980505138
```

Рис. 2.10: Евклидова и p- нормы; Евклидово расстояние

```
[20]: # Проверка по базовому определению:
println("Проверка по базовому определению = ", sqrt(sum((X-Y).^2)))
# Угол между двумя векторами:
println("Угол между двумя векторами = ", acos((transpose(X)*Y)/(norm(X)*norm(Y))))
```

```
Проверка по базовому определению = 9.486832980505138
Угол между двумя векторами = 2.4404307889469252
```

Рис. 2.11: Проверка по баз. опред. и угол между векторами

Вычисление нормы для двумерной матрицы:

```
[21]: # Создание матрицы:
d = [5 -4 2 ; -1 2 3; -2 1 0]
# Вычисление Евклидовой нормы:
println("Евклидова норма = ", opnorm(d))
# Вычисление p-нормы:
p=1
println("p-норма = ", opnorm(d,p))
```

```
Евклидова норма = 7.147682841795258
p-норма = 8.0
```

Рис. 2.12: Евклидова и и p- нормы для 2мерной матрицы

```
[23]: # Поворот на 180 градусов:
      rot180(d)
```

```
[23]: 3×3 Matrix{Int64}:
       0   1  -2
       3   2  -1
       2  -4   5
```

Рис. 2.13: Поворот на 180 градусов

```
[25]: # Переворачивание строк:
      reverse(d,dims=1)
```

```
[25]: 3×3 Matrix{Int64}:
       -2   1  0
       -1   2  3
        5  -4  2
```

Рис. 2.14: Переворачивание строк

```
[26]: # Переворачивание столбцов
      reverse(d,dims=2)
```

```
[26]: 3×3 Matrix{Int64}:
       2  -4   5
       3   2  -1
       0   1  -2
```

Рис. 2.15: Переворачивание столбцов

2.4 Матричное умножение, единичная матрица, скалярное произведение

```
[27]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:
      A = rand(1:10,(2,3))
```

```
[27]: 2×3 Matrix{Int64}:
       2  8  4
       2  2  7
```

```
[28]: # Матрица 3x4 со случайными целыми значениями от 1 до 10:
      B = rand(1:10,(3,4))
```

```
[28]: 3×4 Matrix{Int64}:
       4  9  3   6
       1  1  4   8
       8  3  9  10
```

Рис. 2.16: Матрицы со случ. цел. значениями

```
[29]: # Произведение матриц A и B:
      A*B
```

```
[29]: 2×4 Matrix{Int64}:
       48  38  74  116
       66  41  77   98
```

Рис. 2.17: Произведение матриц

```
[30]: # Единичная матрица 3x3:
      Matrix{Int}(I, 3, 3)
```

```
[30]: 3×3 Matrix{Int64}:
       1  0  0
       0  1  0
       0  0  1
```

Рис. 2.18: Единичная матрица

```
[31]: # Скалярное произведение векторов X и Y:
      X = [2, 4, -5]
      Y = [1,-1,3]
      dot(X,Y)

[31]: -17

[32]: # тоже скалярное произведение:
      X'Y

[32]: -17
```

Рис. 2.19: Скалярное произведение векторов X и Y

2.5 Факторизация. Специальные матричные структуры

В математике факторизация (или разложение) объекта — его декомпозиция (например, числа, полинома или матрицы) в произведение других объектов или факторов, которые, будучи перемноженными, дают исходный объект. Матрица может быть факторизована на произведение матриц специального вида для приложений, в которых эта форма удобна. К специальным видам матриц относят ортогональные, унитарные и треугольные матрицы. LU-разложение — представление матрицы A в виде произведения двух матриц L и U, L — нижняя треугольная матрица, а U — верхняя треугольная матрица. LU-разложение существует только в том случае, когда матрица A обратима, а все её ведущие (угловые) главные миноры невырождены. Обращение матрицы A эквивалентно решению линейной системы AX = I, где X — неизвестная матрица, I — единичная матрица. Решение X этой системы является обратной матрицей A−1 . LUP-разложение — представление матрицы A в виде произведения P A = LU, где матрица L является нижнетреугольной с единицами на главной диагонали, U — верхнетреугольная общего вида матрица, P — матрица перестановок, получаемая из единичной матрицы путём перестановки строк или столбцов. QR-разложение матрицы — представление матрицы в виде произведения унитарной (или ортогональной) матрицы Q и верхнетреугольной матрицы R. QR-разложение применяется для

нахождения собственных векторов и собственных значений матрицы. Q является ортогональной матрицей, если Q^TQ = I, где I — единичная матрица. Спектральное разложение матрицы A — представление её в виде произведения A = V ΛV −1, где V — матрица, столбцы которой являются собственными векторами матрицы A, Λ — диагональная матрица с соответствующими собственными значениями на главной диагонали, $V^{-}1$ — матрица, обратная матрице V. Рассмотрим несколько примеров. Для работы со специальными матричными структурами потребуется пакет LinearAlgebra. Решение систем линейный алгебраических уравнений Ax = b:

```
[33]: # Задаём квадратную матрицу 3x3 со случайными значениями:
      A = rand(3, 3)
```

```
[33]: 3×3 Matrix{Float64}:
       0.0549676  0.0761364  0.0328232
       0.233882   0.168049   0.653198
       0.913299   0.934554   0.627835
```

Рис. 2.20: Квадратная матрица 3x3

```
[34]: # Задаём единичный вектор:
      x = fill(1.0, 3)
```

```
[34]: 3-element Vector{Float64}:
       1.0
       1.0
       1.0
```

Рис. 2.21: Единичный вектор

```
[35]: # Задаём вектор b:
      b = A*x
```

```
[35]: 3-element Vector{Float64}:
       0.1639271578901671
       1.0551286182079866
       2.475688944187798
```

Рис. 2.22: Вектор b

```
[36]: # Решение исходного уравнения получаем с помощью функции \
      # (убеждаемся, что x - единичный вектор):
      A\b
```

```
[36]: 3-element Vector{Float64}:
       1.000000000000001
       0.9999999999999988
       1.0
```

Рис. 2.23: Решение исходного уравнения

Julia позволяет вычислять LU-факторизацию и определяет составной тип факторизации для его хранения

```
[37]: # LU-факторизация:
      Alu = lu(A)
```

```
[37]: LU{Float64, Matrix{Float64}, Vector{Int64}}
      L factor:
      3×3 Matrix{Float64}:
       1.0         0.0      0.0
       0.256084    1.0      0.0
       0.0601857  -0.27905  1.0
      U factor:
      3×3 Matrix{Float64}:
       0.913299   0.934554   0.627835
       0.0       -0.0712758  0.492419
       0.0        0.0        0.132446
```

Рис. 2.24: LU-факторизация

Различные части факторизации могут быть извлечены путём доступа к их специальным свойствам:

```
[38]: # Матрица перестановок:
      Alu.P
```

```
[38]: 3×3 Matrix{Float64}:
       0.0  0.0  1.0
       0.0  1.0  0.0
       1.0  0.0  0.0
```

Рис. 2.25: Матрица перестановок

```
[39]: # Вектор перестановок:
      Alu.p

[39]: 3-element Vector{Int64}:
       3
       2
       1
```

Рис. 2.26: Вектор перестановок

```
[40]: # Матрица L:
      Alu.L

[40]: 3×3 Matrix{Float64}:
       1.0         0.0      0.0
       0.256084    1.0      0.0
       0.0601857  -0.27905  1.0
```

Рис. 2.27: Матрица L

```
[41]: # Матрица U:
      Alu.U

[41]: 3×3 Matrix{Float64}:
       0.913299   0.934554   0.627835
       0.0       -0.0712758  0.492419
       0.0        0.0        0.132446
```

Рис. 2.28: Матрица U

Исходная система уравнений □□ = □ может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации:

```
[42]: # Решение СЛАУ через матрицу A:
      A\b

[42]: 3-element Vector{Float64}:
       1.000000000000001
       0.9999999999999988
       1.0
```

Рис. 2.29: Решение СЛАУ через матрицу A

```
[43]: # Решение СЛАУ через объект факторизации:
      Alu\b
```

```
[43]: 3-element Vector{Float64}:
       1.000000000000001
       0.9999999999999988
       1.0
```

Рис. 2.30: Решение через объект факторизации

```
[45]: # Детерминант матрицы A:
      println("Детерминант матрицы A = ", det(A))
      # Детерминант матрицы A через объект факторизации:
      println("Детерминант матрицы A через объект факторизации = ", det(Alu))

      Детерминант матрицы A = 0.008621728983691011
      Детерминант матрицы A через объект факторизации = 0.008621728983691011
```

Рис. 2.31: Детерминант матрицы A

Julia позволяет вычислять QR-факторизацию и определяет составной тип факторизации для его хранения:

```
[46]: # QR-факторизация:
      Aqr = qr(A)
```

```
[46]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
      Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
      R factor:
      3×3 Matrix{Float64}:
       -0.944371  -0.949855   -0.770859
        0.0        0.0721371  -0.460089
        0.0        0.0        -0.126559
```

Рис. 2.32: QR-факторизация

По аналогии с LU-факторизацией различные части QR-факторизации могут быть извлечены путём доступа к их специальным свойствам:

```
[47]: # Матрица Q:
      Aqr.Q

[47]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
```

Рис. 2.33: Матрица Q

```
[48]: # Матрица R:
      Aqr.R

[48]: 3×3 Matrix{Float64}:
       -0.944371  -0.949855   -0.770859
        0.0        0.0721371  -0.460089
        0.0        0.0        -0.126559
```

Рис. 2.34: Матрица R

```
[49]: # Проверка, что матрица Q - ортогональная:
      Aqr.Q'*Aqr.Q

[49]: 3×3 Matrix{Float64}:
        1.0          5.55112e-17  -1.11022e-16
        5.55112e-17  1.0          -5.55112e-17
       -2.22045e-16  5.55112e-17   1.0
```

Рис. 2.35: Проверка, что матрица Q - ортогональная

Примеры собственной декомпозиции матрицы A:

```
[50]: # Симметризация матрицы A:
      Asym = A + A'

[50]: 3×3 Matrix{Float64}:
       0.109935  0.310018  0.946122
       0.310018  0.336098  1.58775
       0.946122  1.58775   1.25567
```

Рис. 2.36: Симметризация матрицы A

```
[51]: # Спектральное разложение симметризованной матрицы:
      AsymEig = eigen(Asym)
```

```
[51]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
      values:
      3-element Vector{Float64}:
       -0.9806802197717008
       -0.1022174280243332
        2.7846017307313895
      vectors:
      3×3 Matrix{Float64}:
       -0.357957   0.871427   -0.335382
       -0.683191  -0.489268   -0.542094
        0.636488  -0.0350837  -0.770489
```

Рис. 2.37: Спектральное разложение симметризованной матрицы

```
[52]: # Собственные значения:
      AsymEig.values
```

```
[52]: 3-element Vector{Float64}:
       -0.9806802197717008
       -0.1022174280243332
        2.7846017307313895
```

Рис. 2.38: Собственные значения

```
[53]: #Собственные векторы:
      AsymEig.vectors
```

```
[53]: 3×3 Matrix{Float64}:
       -0.357957   0.871427   -0.335382
       -0.683191  -0.489268   -0.542094
        0.636488  -0.0350837  -0.770489
```

Рис. 2.39: Собственные векторы

```
[54]: # Проверяем, что получится единичная матрица:
      inv(AsymEig)*Asym
```

```
[54]: 3×3 Matrix{Float64}:
       1.0          6.66134e-16  9.99201e-16
       9.99201e-16  1.0          5.55112e-16
       8.88178e-16  1.38778e-15  1.0
```

Рис. 2.40: Проверка, что получится единичная матрица

Далее рассмотрим примеры работы с матрицами большой размерности и специальной структуры.

```
[55]: # Матрица 1000 x 1000:
      n = 1000
      A = randn(n,n)
      # Симметризация матрицы:
      Asym = A + A'
```

```
[55]: 1000×1000 Matrix{Float64}:
        2.78491    -3.19685    0.138315   …  -0.362048  -0.0809649  -2.16464
       -3.19685     0.4048     0.573358      -0.705786  -0.336041   -1.38256
        0.138315    0.573358   1.9574         0.357446  -2.75449    -0.609017
        0.425647    0.34514    0.0698925     -0.541615  -1.67797     0.569073
        2.86042     0.369735  -1.98158       -1.12937   -0.414302    0.039149
        0.461048    0.262558  -2.03391    …  -2.11652   -1.10576     0.750604
       -1.282       0.671496  -0.209318       0.025468  -1.84229     0.975415
       -1.84529    -0.575317  -2.31002       -1.76062   -1.93665    -2.0774
        0.0707416  -3.44068   -0.754021      -0.45399   -1.44262     0.0357614
       -0.908996   -0.679798   1.25434        1.30907    0.724121    2.0947
       -1.27428     1.26423   -0.449031   …   0.909594   1.85016    -2.29687
        0.0431969  -2.07272   -0.988403       0.638176   3.09064     1.47555
        0.340624    0.452424   2.54543        1.16911    3.80136     1.1884
        ⋮                                  ⋱
       -1.17549    -0.327251   1.32091       -2.03629    1.74096     2.67405
        0.395764    1.01168   -1.07389        2.34125   -0.48007     0.802822
        0.270349   -0.580772  -1.22026    …  -4.16321    0.705047   -1.51225
        0.364096   -1.51081   -0.577714       0.328006  -1.89059    -1.6458
        3.53659    -0.879684   1.44238        0.108834  -0.985793   -1.05202
        4.00736     0.822713   0.470554      -1.92585   -1.58345     0.0938736
        0.255634    3.74543    0.142054       0.476258   0.277889    0.738295
       -0.875156   -0.765271   0.941752   …   0.293681  -1.25882     1.61734
        1.41395    -1.87126   -1.20024       -0.836433   1.7735     -1.27582
       -0.362048   -0.705786   0.357446       0.557339  -0.97328    -0.883924
       -0.0809649  -0.336041  -2.75449       -0.97328   -2.81736    -0.40063
       -2.16464    -1.38256   -0.609017      -0.883924  -0.40063    -2.95445
```

Рис. 2.41: Симметризация матрицы

```
[56]: # Проверка, является ли матрица симметричной:
      issymmetric(Asym)
```

```
[56]: true
```

Рис. 2.42: Проверка симметричности

Пример добавления шума в симметричную матрицу (матрица уже не будет симметричной):

```
[57]: # Добавление шума:
      Asym_noisy = copy(Asym)
      Asym_noisy[1,2] += 5eps()

[57]: -3.196846840595434

[58]: # Проверка, является ли матрица симметричной:
      issymmetric(Asym_noisy)

[58]: false
```

Рис. 2.43: Добавление шума и проверка симметр-ти

В Julia можно объявить структуру матрица явно, например, используя Diagonal, Triangular, Symmetric, Hermitian, Tridiagonal и SymTridiagonal:

```
[59]: # Явно указываем, что матрица является симметричной:
      Asym_explicit = Symmetric(Asym_noisy)
```

```
[59]: 1000×1000 Symmetric{Float64, Matrix{Float64}}:
        2.78491    -3.19685    0.138315   …  -0.362048  -0.0809649  -2.16464
       -3.19685     0.4048     0.573358      -0.705786  -0.336041   -1.38256
        0.138315    0.573358   1.9574         0.357446  -2.75449    -0.609017
        0.425647    0.34514    0.0698925     -0.541615  -1.67797     0.569073
        2.86042     0.369735  -1.98158       -1.12937   -0.414302    0.039149
        0.461048    0.262558  -2.03391    …  -2.11652   -1.10576     0.750604
       -1.282       0.671496  -0.209318       0.025468  -1.84229     0.975415
       -1.84529    -0.575317  -2.31002       -1.76062   -1.93665    -2.0774
        0.0707416  -3.44068   -0.754021      -0.45399   -1.44262     0.0357614
       -0.908996   -0.679798   1.25434        1.30907    0.724121    2.0947
       -1.27428     1.26423   -0.449031   …   0.909594   1.85016    -2.29687
        0.0431969  -2.07272   -0.988403       0.638176   3.09064     1.47555
        0.340624    0.452424   2.54543        1.16911    3.80136     1.1884
        ⋮                                  ⋱
       -1.17549    -0.327251   1.32091       -2.03629    1.74096     2.67405
        0.395764    1.01168   -1.07389        2.34125   -0.48007     0.802822
        0.270349   -0.580772  -1.22026    …  -4.16321    0.705047   -1.51225
        0.364096   -1.51081   -0.577714       0.328006  -1.89059    -1.6458
        3.53659    -0.879684   1.44238        0.108834  -0.985793   -1.05202
        4.00736     0.822713   0.470554      -1.92585   -1.58345     0.0938736
        0.255634    3.74543    0.142054       0.476258   0.277889    0.738295
       -0.875156   -0.765271   0.941752   …   0.293681  -1.25882     1.61734
        1.41395    -1.87126   -1.20024       -0.836433   1.7735     -1.27582
       -0.362048   -0.705786   0.357446       0.557339  -0.97328    -0.883924
       -0.0809649  -0.336041  -2.75449       -0.97328   -2.81736    -0.40063
       -2.16464    -1.38256   -0.609017      -0.883924  -0.40063    -2.95445
```

Рис. 2.44: Явное указание симметричности

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом BenchmarkTools:

```
[62]: import Pkg
      Pkg.add("BenchmarkTools")
      using BenchmarkTools
      # Оценка эффективности выполнения операции по нахождению
      # собственных значений симметризованной матрицы:
      @btime eigvals(Asym);
```

```
  Resolving package versions...
    Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
   Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
65.920 ms (18 allocations: 7.98 MiB)
```

Рис. 2.45: Оценка эффективности для симметризованной матрицы

```
[63]: # Оценка эффективности выполнения операции по нахождению
      # собственных значений зашумлённой матрицы:
      @btime eigvals(Asym_noisy);
```

```
357.488 ms (27 allocations: 7.93 MiB)
```

Рис. 2.46: Оценка эффективности для зашумлённой матрицы

```
[64]: # Оценка эффективности выполнения операции по нахождению
      # собственных значений зашумлённой матрицы,
      # для которой явно указано, что она симметричная:
      @btime eigvals(Asym_explicit);
```

```
67.992 ms (18 allocations: 7.98 MiB)
```

Рис. 2.47: Оценка эффективности для сим. зашумлённой матрицы

Далее рассмотрим примеры работы с разряженными матрицами большой размерности. Использование типов Tridiagonal и SymTridiagonal для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами:

```
[65]: # Трёхдиагональная матрица 1000000 x 1000000:
      n = 1000000;
      A = SymTridiagonal(randn(n), randn(n-1))
```

```
[65]: 1000000×1000000 SymTridiagonal{Float64, Vector{Float64}}:
 -1.066     2.59106     ⋅          …    ⋅          ⋅          ⋅
  2.59106   0.165876  -0.715324         ⋅          ⋅          ⋅
   ⋅       -0.715324   0.0905971        ⋅          ⋅          ⋅
   ⋅         ⋅        -0.394007         ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅          …    ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅          …    ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
  ⋮                                  ⋱
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅          …    ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅               ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅          …    ⋅          ⋅          ⋅
   ⋅         ⋅          ⋅            -1.0367       ⋅          ⋅
   ⋅         ⋅          ⋅             2.25571   -0.912756     ⋅
   ⋅         ⋅          ⋅            -0.912756   1.10327   1.70512
   ⋅         ⋅          ⋅              ⋅         1.70512   0.183777
```

Рис. 2.48: Трёхдиагональная матрица

```
[66]: # Оценка эффективности выполнения операции по нахождению
      # собственных значений:
      @btime eigmax(A)

        325.920 ms (39 allocations: 183.11 MiB)

[66]: 6.212407628995837
```

Рис. 2.49: Оценка эффективности выполнения

При попытке задать подобную матрицу обычным способом и посчитать её собственные значения, вы скорее всего получите ошибку переполнения памяти.

2.6 Общая линейная алгебра

Обычный способ добавить поддержку числовой линейной алгебры - это обернуть подпрограммы BLAS и LAPACK. Собственно, для матриц с элементами Float32,Float64, Complex {Float32} или Complex {Float64} разработчики Julia использовали такое же решение. Однако Julia также поддерживает общую линейную алгебру, что позволяет, например, работать с матрицами и векторами рациональных чисел. Для задания рационального числа используется двойная косая черта: 1//2 В следующем примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем BigInt):

```
[67]: # Матрица с рациональными элементами:
      Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10

[67]: 3×3 Matrix{Rational{BigInt}}:
        1    1//10  3//10
       1//2  1//5   3//10
       2//5  9//10  4//5
```

Рис. 2.50: Матрица с рациональными элементами

```
[68]: # Единичный вектор:
      x = fill(1, 3)

[68]: 3-element Vector{Int64}:
       1
       1
       1
```

Рис. 2.51: Единичный вектор

```
[69]: # Задаём вектор b:
      b = Arational*x
```

```
[69]: 3-element Vector{Rational{BigInt}}:
        7//5
         1
       21//10
```

Рис. 2.52: Вектор b

```
[70]: # Решение исходного уравнения получаем с помощью функции \
      # (убеждаемся, что x - единичный вектор):
      Arational\b
```

```
[70]: 3-element Vector{Rational{BigInt}}:
       1
       1
       1
```

Рис. 2.53: Решение с помощью функции \

```
[72]: # LU-разложение:
      lu(Arational)
```

```
[72]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
      L factor:
      3×3 Matrix{Rational{BigInt}}:
        1       0      0
       2//5     1      0
       1//2   15//86   1
      U factor:
      3×3 Matrix{Rational{BigInt}}:
       1    1//10    3//10
       0   43//50   17//25
       0     0      27//860
```

Рис. 2.54: LU-разложение

2.7 Задания для самостоятельного выполнения

1.

```
[73]: v = [1, 2, 3]
      dot_v = dot(v, v)
      println("Скалярное произведение v·v: ", dot_v)

      Скалярное произведение v·v: 14
```

Рис. 2.55: Задание №1

2.

```
[75]: outer_v = v * v'
      outer_v

[75]: 3×3 Matrix{Int64}:
       1  2  3
       2  4  6
       3  6  9
```

Рис. 2.56: Задание №2

3.

```
[76]: A = [1  1;
           1 -1]
      b = [2; 3]

      solution = A \ b
      println("Решение (x, y): ", solution)

      Решение (x, y): [2.5, -0.5]
```

Рис. 2.57: Задание №3.a

```
[83]: A = [1  1;
           2  2]
      b = [2; 4]

      try
          solution = A \ b
          println("Решение (x, y): ", solution)
      catch e
          println("Система несовместна или вырождена: ", e)
      end
```

```
Система несовместна или вырождена: SingularException(2)
```

Рис. 2.58: Задание №3.b

4.

```
[79]: using LinearAlgebra

      A = [1 1 1;
           1 -1 -2]
      b = [2; 3]

      try
          solution = A \ b
          println("Решение (x, y, z): ", solution)
      catch e
          println("Система недоопределена: бесконечно много решений или противоречие.")
      end
```

```
Решение (x, y, z): [2.2142857142857144, 0.35714285714285704, -0.5714285714285712]
```

Рис. 2.59: Задание №4.a

```
[80]: A = [1  1  1;
           2  2 -3;
           3  1  1]
      b = [2; 4; 1]

      solution = A \ b
      println("Решение (x, y, z): ", solution)
```

```
Решение (x, y, z): [-0.5, 2.5, 0.0]
```

Рис. 2.60: Задание №4.b

```
[81]: A = [1  1  1;
           1  1  2;
           2  2  3]
      b = [1; 0; 1]

      try
          solution = A \ b
          println("Решение (x, y, z): ", solution)
      catch e
          println("Система несовместна или вырождена: ", e)
      end
```

```
Система несовместна или вырождена: SingularException(2)
```

Рис. 2.61: Задание №4.c

```
[82]: A = [1  1  1;
           1  1  2;
           2  2  3]
      b = [1; 0; 0]
      try
          solution = A \ b
          println("Решение (x, y, z): ", solution)
      catch e
          println("Система несовместна или вырождена: ", e)
      end
```

```
Система несовместна или вырождена: SingularException(2)
```

Рис. 2.62: Задание №4.d

5.

```
[88]: using LinearAlgebra
A = [1 -2;
     -2 1]
# находим собственные значения и собственные векторы
eigen_result = eigen(A)
# извлекаем диагональную матрицу D (собственные значения на диагонали)
D = Diagonal(eigen_result.values)
```

```
[88]: 2×2 Diagonal{Float64, Vector{Float64}}:
 -1.0   ⋅
  ⋅    3.0
```

Рис. 2.63: Задание №5

```
[87]: using LinearAlgebra
A = [1 -2;
     -2 3]
# находим собственные значения и векторы
eigen_result = eigen(A)
# формируем диагональную матрицу D из собственных значений
D = Diagonal(eigen_result.values)
D
```

```
[87]: 2×2 Diagonal{Float64, Vector{Float64}}:
 -0.236068   ⋅
  ⋅         4.23607
```

Рис. 2.64: Задание №5

6.

```
[89]: using LinearAlgebra
A = [1 -2; -2 1]
A_10 = A^10
A_10
```

```
[89]: 2×2 Matrix{Int64}:
  29525  -29524
 -29524   29525
```

Рис. 2.65: Задание №6.а

```
[90]: using LinearAlgebra
B = [5 -2; -2 5]
# находим собственные значения и векторы
eigen_result = eigen(B)
# извлекаем собственные значения и матрицу собственных векторов
eigenvals = eigen_result.values
eigenvecs = eigen_result.vectors
# вычисляем квадратный корень из собственных значений
sqrt_eigenvals = sqrt.(eigenvals)
# строим диагональную матрицу из квадратных корней собственных значений
D_sqrt = Diagonal(sqrt_eigenvals)
B_sqrt = eigenvecs * D_sqrt * inv(eigenvecs)
B_sqrt
```

```
[90]: 2×2 Matrix{Float64}:
  2.1889   -0.45685
 -0.45685   2.1889
```

Рис. 2.66: Задание №6.b

```
[91]: C = [1 -2; -2 1]
eigen_result_C = eigen(C)  # находим собственные значения и векторы
eigenvals_C = eigen_result_C.values  # собственные значения
eigenvecs_C = eigen_result_C.vectors  # матрица собственных векторов
# вычисляем кубический корень из собственных значений
cbrt_eigenvals_C = cbrt.(eigenvals_C)
# строим диагональную матрицу из кубических корней собственных значений
D_cbrt_C = Diagonal(cbrt_eigenvals_C)
# восстанавливаем кубический корень из матрицы:
C_cbrt = eigenvecs_C * D_cbrt_C * inv(eigenvecs_C)
C_cbrt
```

```
[91]: 2×2 Matrix{Float64}:
  0.221125  -1.22112
 -1.22112    0.221125
```

Рис. 2.67: Задание №6.c

7.

```
[93]: using LinearAlgebra
A = [140  97  74  168  131;
     97 106  89  131   36;
     74  89 152  144   71;
    168 131 144   54  142;
    131  36  71  142   36]
# Находим собственные значения матрицы A
eigen_result = eigen(A)
eigenvalues = eigen_result.values
println("Собственные значения матрицы A:")
println(eigenvalues)
println()
# Создаём диагональную матрицу из собственных значений
D = Diagonal(eigenvalues)
println("Диагональная матрица D (из собственных значений):")
println(D)
println()
# Создаём нижнедиагональную матрицу из A
LowerTriangular_A = LowerTriangular(A)
# преобразуем в обычную матрицу для вывода (без специальных флагов)
LowerTriangular_matrix = Matrix(LowerTriangular_A)
println("Нижнедиагональная матрица (элементы выше главной диагонали обнулены):")
println(LowerTriangular_matrix)
println()
# Оцениваем эффективность операций
# Время вычисления собственных значений
@time eigen(A);
# Время создания диагональной матрицы
@time Diagonal(eigenvalues);
# Время создания нижнедиагональной матрицы
@time LowerTriangular(A);
println("Эффективность операций:")
println("Вычисление собственных значений (eigen): время выполнения измерено с помощью @time")
println("Создание диагональной матрицы (Diagonal): быстро, так как работает с вектором")
println("Создание нижнедиагональной матрицы (LowerTriangular): эффективно, так как Julia использует оптимизированные операции с треугольными матрицами")
```

Рис. 2.68: Задание №7

```
Собственные значения матрицы A:
[-128.49322764802145, -55.887784553057, 42.752167279318854, 87.16111477514488, 542.467730146614]

Диагональная матрица D (из собственных значений):
Diagonal([-128.49322764802145, -55.887784553057, 42.752167279318854, 87.16111477514488, 542.467730146614])

Нижнедиагональная матрица (элементы выше главной диагонали обнулены):
[140 0 0 0 0; 97 106 0 0 0; 74 89 152 0 0; 168 131 144 54 0; 131 36 71 142 36]

  0.000046 seconds (15 allocations: 2.688 KiB)
  0.000008 seconds (1 allocation: 16 bytes)
  0.000008 seconds (1 allocation: 16 bytes)
Эффективность операций:
Вычисление собственных значений (eigen): время выполнения измерено с помощью @time
Создание диагональной матрицы (Diagonal): быстро, так как работает с вектором
Создание нижнедиагональной матрицы (LowerTriangular): эффективно, так как Julia использует оптимизированные операции с треугольными матрицами
```

Рис. 2.69: Задание №7

8.

```
[96]: using LinearAlgebra
A_a = [1 2; 3 4]
A_b = (1/2) * [1 2; 3 4]
A_c = (1/10) * [1 2; 3 4]

# Функция для проверки продуктивности матрицы
function is_productive(A)
    n = size(A, 1)
    E = Matrix{Float64}(I, n, n)  # единичная матрица того же размера
    A_diff = E - A
    # Проверяем, существует ли обратная матрица
    if det(A_diff) == 0
        return false, "Матрица (E - A) вырождена, обратная не существует."
    end
    A_inv = inv(A_diff)  # вычисляем (E - A)⁻¹
    # Проверяем, все ли элементы обратной матрицы неотрицательны
    if all(x >= 0 for x in A_inv)
        return true, A_inv
    else
        return false, "Элементы обратной матрицы (E - A)⁻¹ отрицательны."
    end
end
```

Рис. 2.70: Задание №8.а

```julia
# Проверяем продуктивность каждой матрицы
println("а) Матрица A:")
result_a, info_a = is_productive(A_a)
println("Продуктивна: ", result_a)
if !result_a
    println("Причина: ", info_a)
else
    println("Обратная матрица (E - A)⁻¹: ", info_a)
end
println()

println("б) Матрица (1/2)A:")
result_b, info_b = is_productive(A_b)
println("Продуктивна: ", result_b)
if !result_b
    println("Причина: ", info_b)
else
    println("Обратная матрица (E - A)⁻¹: ", info_b)
end
println()

println("в) Матрица (1/10)A:")
result_c, info_c = is_productive(A_c)
println("Продуктивна: ", result_c)
if !result_c
    println("Причина: ", info_c)
else
    println("Обратная матрица (E - A)⁻¹: ", info_c)
end
```

Рис. 2.71: Задание №8.b

```
а) Матрица A:
Продуктивна: false
Причина: Элементы обратной матрицы (E - A)⁻¹ отрицательны.

б) Матрица (1/2)A:
Продуктивна: false
Причина: Элементы обратной матрицы (E - A)⁻¹ отрицательны.

в) Матрица (1/10)A:
Продуктивна: true
Обратная матрица (E - A)⁻¹: [1.25 0.4166666666666667; 0.6250000000000001 1.875]
```

Рис. 2.72: Задание №8.c

3 Выводы

В результате выполнения лабораторной работы, я изучил возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.